

Running the workflow and workflow start and testing commands

Command to show

Before starting the listener, you must do this in n8n:

Open workflow

Click Webhook node

Click:

"Listen for test event"

Keep tab open

Then on the VM:

start give load... and top... (Bottom command there) Monitoring is currently evaluating the VM load condition

URL who listen

Invoke-WebRequest `Uri "<https://dt-iwpb-platform-dev.gcp.cloud.uk.hsbc/webhook-test/snowbridge-sse-alert>"

keep CPU stress running if you want real monitoring detection
or generate another controlled issue

Then start:

python3 sse_api_vm.py

What you expect to happen:

Snowbridge emits a live Nebula alert
Python listener receives it
it forwards to webhook-test
n8n workflow triggers
nodes animate green live

=====

PUT load on VM for test

On the GCP VM, first identify CPU count:

nproc

Then generate CPU load for a few minutes.

yes > /dev/null &

That starts one busy CPU thread.

To create heavier load:

for i in {1..4}; do yes > /dev/null & done

Adjust the number based on VM size.

Then verify: `

Method POST `

(Command at bottom)

top

After the demo, stop the load:

pkill yes

If ALERT matches for Outlook.

ps -fp 261845

ps -ef | grep sse_api_vm.py

=====

What curl actually does

When you run a curl command against the webhook URL, you are doing exactly what the SSE listener does when it receives a real alert from Snowbridge — posting a JSON payload to n8n's webhook endpoint. The workflow cannot tell the difference. It processes the payload the same way regardless of whether it came from a real GMOM event or a manually crafted curl. The reason we use curl for demos is control. A real alert arrives when GMOM decides to send one. A curl arrives the moment you press Enter. For a timed demo where you need to show a specific platform type or a specific metric, curl gives you that certainty.

=====

First time running

Invoke-WebRequest `

-Uri "<https://dt-iwpb-platform-dev.gcp.cloud.uk.hsbc/webhook-test/snowbridge-sse-alert>" `

-Method POST

-ContentType "application/json" `

-Body '{

"ticket_type": "alert",

"event_id": 999999920,

"data": {

"assignment_group": "GDT-WPB-DSP-NEBULA",

"cmdb_ci": "l1trriage-rag-api",

"environment": "hsbcitid",

"severity": "Critical",

"metric_name": "CPU",

"description": "DEMO CPU spike alert",

"source": "GMOM",

"state": "Open"

},

"snow_env": "hsbcitid"

}'

====

This is exmaple to trigger create real VM load,but simultaneously keep a backup manual webhook payload ready.

=====

Powershell command to trigger: **CPU**

Invoke-WebRequest -Uri "<https://dt-iwpb-platform-dev.gcp.cloud.uk.hsbc/webhook/snowbridge-sse-alert>" -Method POST -ContentType "application/json" -UseBasicParsing -Body '{"ticket_type":"alert","event_id":999999920,"data":{"assignment_group":"GDT-WPB-DSP-NEBULA","cmdb_ci":"l1trriage-rag-api","environment":"hsbcitid","severity":"Critical","metric_name":"CPU","description":"DEMO CPU spike alert","source":"GMOM","state":"Open"},"snow_env":"hsbcitid"}'

=====

MEMORY ALERT

Load

MEMORY ALERT

Consume RAM aggressively:

Start The Memory Load (Create The Problem)

"`

```
python3 -c "a=['A'*1024*1024 for _ in range(1500)]; input('Memory load running...')"
```

Leave this running. Do not press Enter yet.

Open a second terminal to the VM and run:

free-m

top

Show them memory is now under pressure: simulated a memory exhaustion event.

Fire the alert from command prompt.

This allocates roughly ~1.5 GB RAM.

Stop with: Ctrl + C

Powershell command to trigger: Memory

```
Invoke-WebRequest -Uri "https://dt-iwpb-platform-dev.gcp.cloud.uk.hsbc/webhook/snowbridge-sse-alert" -Method POST -ContentType "application/json" -UseBasicParsing -Body '{"ticket_type":"alert","event_id":9999999921,"data":{"assignment_group":"GDT-WPB-DSP-NEBULA","cmdb_ci":"l1triage-rag-api","environment":"hsbcid","severity":"Critical","metric_name":"Memory","description":"DEMO Memory exhaustion detected","source":"GMOM","state":"Open"},"snow_env":"hsbcid"}
```

Show The Email Arriving

Cause: PID [number] (python3 -c a=...) was using ~49% of RAM Analysis: Host had only 68 MiB free, swap at 900 MiB, OOM kill risk elevated

Action: Stop the ad hoc Python memory-load process

"The AI identified the exact process, the exact PID, the exact memory percentage. No human looked at this VM. This happened automatically in under 3 minutes."

====

APPLICATION ALERT

Load

Start CPU Load In Background First

On the VM run this:

```
python3 -c "while True: pass" &
```

The puts it in the background so it keeps running.

Note the PID it gives you:

[1] 325847 this is your PID, note it down

You should see:

```
325847 bharat_ + 20 0 ... 99.0 0.0 python3
```

"CPU is now being consumed by this Python process. Watch what happens."

Also Kill The App To Make Application Alert Real

```
pkill -f rag_api_server_stdlib.py
```

Then confirm it is gone:

```
ps aux | grep rag_api
```

Run high CPU Python app loop:

```
python3 -c "while True: pass"
```

Fire The Alert-

```
Invoke-WebRequest -Uri "https://dt-iwpb-platform-dev.gcp.cloud.uk.hsbc/webhook/snowbridge-sse-alert" -Method POST -ContentType "application/json" -UseBasicParsing -Body '{"ticket_type":"alert","event_id":9999999925,"data":{"assignment_group":"GDT-WPB-DSP-NEBULA","cmdb_ci":"l1triage-rag-api","environment":"hsbcid","severity":"Critical","metric_name":"Application","description":"DEMO Application process down","source":"GMOM","state":"Open"},"snow_env":"hsbcid"}'
```

stop your app process

```
pkill-f rag_api_server_stdlib.py
```

```
===
```

```
kill 325847
```

```
pkill -f "while True"
```

```
top -bn1 | head -5
```

```
=====
```

NETWORK ALERT

Start Network Load On VM

Open VM terminal and run one of these:

Option A — continuous download (best for showing traffic)

```
wget -O /dev/null https://speed.hetzner.de/1GB.bin
```

Or if wget is blocked on HSBC network:

Option B — continuous ping

```
ping google.com
```

Option C — show current connections

```
watch-n1 'ss -tunap | head -20'
```

Step 2 — Show Network Activity Before Alert

```
"`
```

On VM in a second terminal:

```
ss -tunap | head -20
```

Show active connections.

Also run:

```
ip addr
```

```
ip route
```

This is the network state right now. Active connections, IP configuration, routing table.

Step 3 — Fire The Alert

Generate network traffic externally.

```
ping google.com
```

```
curl -L https://speed.hetzner.de/1GB.bin -o /dev/null
```

```
wget -O /dev/null https://speed.hetzner.de/1GB.bin-
```

```
--
```

```
Invoke-WebRequest -Uri "https://dt-iwpb-platform-dev.gcp.cloud.uk.hsbc/webhook/snowbridge-sse-alert" -Method POST -ContentType "application/json" -UseBasicParsing -Body '{"ticket_type":"alert","event_id":9999999926,"data":{"assignment_group":"GDT-WPB-DSP-NEBULA","cmdb_ci":"l1triage-rag-api","environment":"hsbcid","severity":"Critical","metric_name":"Network","description":"DEMO Network packet loss detected","source":"GMOM","state":"Open"},"snow_env":"hsbcid"}'
```

```
===
```

BIGQUERY ALERT

```
load
```

```
while true; do
```

```
  bq query --use_legacy_sql=false 'SELECT COUNT(*) FROM `project.dataset.large_table`'
```

```
done
```

force failing queries:

```
bq query --use_legacy_sql=false 'SELECT * FROM nonexistent.dataset.table'
```

This may trigger query failure monitoring.
CPU ALERT (your current working method)

```
yes > /dev/null &  
for i in {1..4}; do yes > /dev/null & done
```

```
Stop  
killall yes
```

-

Test 1 — BigQuery with real L1_ALERT_AUTOMATION dataset

```
Invoke-WebRequest -Uri "https://dt-iwpb-platform-dev.gcp.cloud.uk.hsbc/webhook/snowbridge-sse-alert" -Method POST -ContentType "application/json" -UseBasicParsing -Body '{"ticket_type":"alert","event_id":  
9999999924,"data":{"assignment_group":"GDT-WPB-DSP-NEBULA","cmdb_ci":"bigquery-analytics-prod","environment":"hsbcitid","severity":"Critical","metric_name":"BigQuery","description":"DEMO BigQuery  
job failure","source":"GMOM","state":"Open"},"snow_env":"hsbcitid"}
```

=====

```
# CLOUD RUN ALERT  
LOAD
```

```
while true; do  
  curl -s  
https://YOUR-CLOUDRUN-URL  
done
```

Parallel version:

```
for i in {1..20}; do  
  while true; do  
    curl -s https://bash  
  for i in {1..20}; do  
    while true; do  
      curl -s  
    > /dev/null  
https://YOUR-CLOUDRUN-URL  
    done &  
  done  
  > /dev/null  
" `
```

Stop all curls:

```
pkill curl
```

==

Test 2 — Cloud Run with real supersetcloudrun service

```
Invoke-WebRequest -Uri "https://dt-iwpb-platform-dev.gcp.cloud.uk.hsbc/webhook/snowbridge-sse-alert" -Method POST -ContentType "application/json" -UseBasicParsing -Body '{"ticket_type":"alert","event_id":  
9999999923,"data":{"assignment_group":"GDT-WPB-DSP-NEBULA","cmdb_ci":"nebula-api-service","environment":"hsbcitid","severity":"Critical","metric_name":"Cloud_Run","description":"DEMO Cloud Run  
error rate spike","source":"GMOM","state":"Open"},"snow_env":"hsbcitid"}
```

=====